

POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Executive Summary of the Thesis

Towards fast and side-channel resistant implementations of Hamming Quasi-Cyclic: a case study on ARM Cortex-M4 devices

Laurea Magistrale in Computer Engineering - Ingegneria Informatica

Author: Domenico Cacace

Advisor: Prof. Alessandro Barenghi

Co-advisor: Prof. Gerardo Pelosi

Academic year: 2022-2023

1. Introduction

The advancements in quantum computing threaten the *status quo* in the cryptography field: a quantum computer with a sufficient amount of qubits is able, thanks to Shor's algorithm, to break most of the current public key cryptosystems in polynomial time.
For this reason in 2016 the National Institute of Standards and Technologies (NIST) launched a competition with the aim of developing and standardizing one or more quantum-resistant cryptosystems. After three rounds CRYSTALS-Kyber, a lattice-based cryptosystem, was selected as standardization candidate; however, in the event of a potentially disruptive attack on lattice-based cryptosystems, NIST decided to further extend the competition selecting four more candidates that rely on different hard problems. Excluding SIKE, the remaining candidates, namely BIKE [1], HQC [10] and Classic McEliece [3] are all based on the hardness of the syndrome decoding problem.
The object of this work is HQC, a code-based cryptosystem whose security relies on the syndrome decoding problem for quasi-cyclic codes. HQC comprises both a Public Key Encryption (PKE) scheme and a Key Encapsulation Mechanism (KEM), obtained by applying the HHK Fujisaki-Okamoto transformation on the former.

Related work Given the sound security record of HQC, much attention has been devoted to its side-channel resistance. Most of the independent analyses focus on extracting information from electromagnetic and power traces, whilst at the end of Round 2 of the NIST contest the authors in [13] and [11] proposed a timing attack that takes advantage of the non constant-time implementation of the BCH decoder; these attacks are no longer relevant, as the authors of HQC swapped the BCH code with a Reed-Muller Reed-Solomon concatenated code, implementing a constant-time decoder. Nontheless, in [6] the authors propose a new timing attack, based on a weakness in the *rejection sampling* procedure, employed to generate random sparse polynomials; as a countermeasure, a more care careful implementation of the rejection sampling procedure has been implemented in Round 4, fixing the issue.
To date, no dedicated constant-time implementation of HQC targeting embedded systems has been proposed, despite the fact that side channel attacks are easier to carry on such platforms.

The current state of software implementations of HQC tailored for the ARM Cortex-M4 microcontroller, chosen by the NIST as the reference platform for microcontroller benchmarks, includes only the one in PQM4 [7], a third-party public project that collects the implementations of NIST Round 4 candidates tailored for Cortex-M4. The latest version of HQC in PQM4 is directly derived from the Round 3 reference implementation, with no optimizations: as a consequence, this implementation is not constant-time and vulnerable to the timing attack presented in [6].

Our contribution In this work we present the first fully constant-time software implementation of HQC, together with a comparative analysis of constant-time binary polynomial multiplication techniques, both tailored for ARM Cortex-M4 microprocessors. We provide an experimental validation of the constant-time properties via a battery of statistical t-tests, while varying the CPU frequency to highlight the stability of our solution across different memory access latencies. Our results show that the optimal solution for the target platform is a FFT-based approach, in contrast with classic divide-et-impera techniques (*i.e.* Karatsuba) which provide better performance on x86 architectures.
Moreover, to counter power attacks, we propose a masking scheme that can be applied to the entire cryptosystem, presenting our results on its potential efficacy. Finally, we compare our work with the latest implementations available of BIKE and Classic McEliece, showing the high efficiency of our approaches.

2. Background

In this section we lay the theoretical foundations required to understand our work. To avoid confusion, throughout this work we will use the following notation, unless specified otherwise. Capital bold letters $(\mathbf{A}, \mathbf{B}, \cdots)$ denote generic sets, while lowercase bold letters $(\mathbf{v}, \mathbf{w}, \cdots)$ denote vectors. Binary Galois fields of dimension n are denoted by $\mathbb{F}_{2^n}$ or $GF(2^n)$, while the vector space of dimension n over the field $GF(2)$ is denoted by either $\mathbb{F}_2^n$ or $\mathcal{V}$. The ring of binary polynomials modulo $(x^n - 1)$ is denoted by $\mathcal{R}$; the value of n is chosen in such a way that the algebra in the ring $\mathcal{R}$ is isomorphic to the algebra of binary $n \times n$ circulant matrices.

2.1. Polynomial multiplication

The most expensive operation in terms of execution time in our cryptosystem is, by far, the polynomial multiplication: it is thus essential to choose a fast algorithm in order to increase the overall performance.

2.1.1 Karatsuba Multiplication

One of the most known fast multiplication algorithms is Karatsuba. This is a divide-et-impera algorithm, which splits the n-degree input polynomials into smaller m-degree ones:

$$a(x) = a_0(x) + x^m \cdot a_1(x)$$

With the operands in this form, it is possible to rewrite the product of two polynomials as

$$c(x) = z_2(x) \cdot x^{2m} + z_1(x) \cdot x^m + z_0(x)$$

with

$$\begin{aligned} z_2(x) &= a_1(x)b_1(x) \\ z_1(x) &= a_1(x)b_0(x) + a_0(x)b_1(x) \\ z_0(x) &= a_0(x)b_0(x) \end{aligned}$$

Karatsuba observed that, at the cost of a few more additions, it is possible to calculate the product using only three multiplications by rewriting z_1 as

$$(a_0(x) + a_1(x))(b_0(x) + b_1(x)) - z_0(x) - z_2(x)$$

This trick allows to calculate the product, under the assumption that $m = \lceil n/2 \rceil$, with $\mathcal{O}(n^{\log_2 3})$ operations.

2.1.2 Frobenius Additive FFT

Instead of multiplying two polynomials (of degree less than n) directly, we can:

- Sample the polynomials at $2n + 1$ distinct points $(a(i), b(i))$
- Perform the pointwise multiplication of the samples $(c(i) = a(i)b(i))$
- Interpolate the points to get the product of the original polynomials

This approach may appear slower than the schoolbook method, as the sampling requires $\mathcal{O}(n^2)$ operations, the pointwise product $\mathcal{O}(n)$

and the interpolation up to $\mathcal{O}(n^3)$ with the Gauss elimination method. However, by recursively splitting the operands and by picking a set of *special* point, such as the n-th roots of unity in $\mathbb{C}$, the sampling and interpolation steps can be performed with the Discrete Fourier Transform and its inverse: algorithms that perform this procedure are commonly called Fast Fourier Transforms (FFTs), and can achieve a complexity of $\mathcal{O}(n \log n)$ operations.
Fields of characteristic two usually do not admit non-trivial roots of unity; nontheless, we can apply the same principle by picking as evaluation points the powers of a primitive element of the field: these algorithms are called Additive FFTs. An example of Additive FFT is the Lin-Chung-Han (LCH) FFT [9] which, for polynomials of degree $n = 2^m$, calculates the product in $\mathcal{O}(n \log n)$.

New polynomial basis In [5] Gao and Mateer use a special basis to represent the polynomials for their FFT. For polynomials of degree $n = 2^m$, the Cantor basis $\mathbf{v} = (v_1, \cdots, v_m)$ of $\mathbb{F}_2^m$ follows the structure:

$$v_1 = 1, \qquad v_i^2 + v_i = v_{i-1} \text{ for } i > 1$$

With respect to this basis we can define a sequence of *nested* subspaces:

$$\mathbf{W}_i := span\{v_0, \cdots, v_{i-1}\}, \quad \mathbf{W}_0 \subset \cdots \subset \mathbb{F}_2^m$$

and, for each subspace, its vanishing polynomial:

$$s_i(x) = \prod_{a \in \mathbf{W}_i} (x - a)$$

For the LCH FFT we first need to represent the polynomials in a different basis, built onto the vanishing polynomials; this basis $(X_1, \cdots, X_{n-1})$ is called *novelpoly basis*, and is defined as:

$$X_k(x) = \prod (s_i(x))^{b_i} \quad \text{with } k = \sum_{i=0}^{\log_2 n} b_i 2^i$$

The butterflies As with other FFT, in the LCH FFT we recursively split the input polynomial into two halves; we also split the set of evaulation points $\mathbf{W}_k$ into two disjoint sets, $\mathbf{W}_{k-1}$ and $\mathbf{W}_k \setminus \mathbf{W_{k-1}} = \{x + v_k | x \in \mathbf{W}_{k-1}\}$. Because of the linearity of the vanishing polynomial $(s_i(x+y) = s_i(x) + s_i(y))$, the evaluations of the input polynomial at $\mathbf{W}_k \setminus \mathbf{W_{k-1}}$ can be quickly computed from the evaluations at $\mathbf{W}_{k-1}$ with the *butterfly* operation, shown at lines 6 and 7 in Algorithm 1

Algorithm 1: Lin-Chung-Han FFT

Input: $f(x)$, in novelpoly basis,
$\alpha \in \mathbb{F}_{2^m}$, a generic field element
Output: $\{f(x) | x \in \mathbf{W}_m\}$

```
1 Function FFT(f(x), α)
2     if deg(f(x)) then
3         return [g_0];
4     k ← max i s.t. 2^i ≤ deg(f(x));
5     g(X) ← p_0(X) + X_{2^k} · p_1(X);
6     h_0(X) ← p_0(X) + s_k(α) · p_1(X);
7     h_1(X) ← h_0(X) + s_k(v_k) · p_1(X);
8     return FFT(h_0(X), α),
          FFT(h_1(X), v_k + α);
```

Notice that the butterfly carries out two field additions but only one field multiplication, because $s_k(v_k) = 1$ in Cantor basis ([4], Appendix A). The output of this algorithm is the set of evaluation points of the input polynomial; these can be multiplied with the evaluations of a second polynomial and, by performing the FFT and basis conversion backwards, we can obtain the product of the two.

Frobenius partition In [12] the authors showed that, by applying the Frobenius map $(\phi : x \mapsto x^2)$, it is possible to derive more values of a polynomial from less evaluation points: given a polynomial $f(x)$ of degree at most n, we can design a set of points $\mathbf{S} \subset \mathbb{F}_{2^m}$ for m power of 2 such that we can derive n evaluations of f from $\mathbf{S}$. In [8] the authors suggest to pick $\mathbf{S}$ as:

$$\mathbf{S} := v_{l+m/2} + \mathbf{W}_l = \{v_{l+m/2} + x | x \in \mathbf{W}_l\}$$

with $l = \log_2 n - \log_2 m$ and size $n' = |\mathbf{S}| = |\mathbf{W}_l| = 2^l = n/m$. As explained by the authors, this construction allows to reach n points in $\mathbb{F}_{2^m}$ by applying the Frobenius map, which also ensures that the evaluation function

$$E_{\mathbf{S}} : f(x) \mapsto f(\mathbf{S}) = \{f(v_{l+m/2+w}) | w \in \mathbf{W}_l\}$$

will be invertible.

2.2. Side-channel attacks

Even though a cryptographic primitive may be secure from a theoretical point of view, its implementation may not. It is possible that a device exposes some *side channels*, observable aspects of a system that may reveal information about the internal state of the system; these side channels, if the device runs a cryptosystem, be exploited to *leak* secret information, such as a message or the keys employed to decrypt it.
Two of the most common side-channel attacks, especially on embedded devices, are:

- **Timing attacks**: the execution time of an operation may depend on the values of its inputs, allowing an attacker to retrieve information by measuring the execution time
- **Power attacks**: the amount of power required by the CPU varies based on the workload, which directly depends on the operations being run and their inputs

The presence of a side channel does not necessarily mean that an attack is possible, as the leaked information may not be sufficient to succesfully exfiltrate information. If this is not the case, there are various countermeasures to mitigate the risk of a successful attack, such as:

- Constant-time algorithms: running an operation takes always the same amount of time, regardless of the input (*e.g.* substitute branching instructions depending on the inputs)
- Masking: randomize the inputs of an algorithm without affecting the outputs; this can be achieved by splitting sensitive data x into $d+1$ *shares*, such that $x = x_0 \oplus \cdots \oplus x_d$, making the sensitive value indistinguishable from random noise.

2.3. Coding theory

To better understand how HQC works, we now give the basic concepts of coding theory.

Definition 2.1 (Circulant matrix)**.** Let $\mathbf{v} = (v_0, \cdots, v_{n-1}) \in \mathcal{V}$. The circulant matrix induced by $\mathbf{v}$ is defined as a matrix whose i-th row is the vector $\mathbf{v}$ shifted by i positions to the right:

$$\mathbf{circ}(\mathbf{v}) = \begin{pmatrix} v_0 & v_1 & \cdots & v_{n-1} \\ v_{n-1} & v_0 & \cdots & v_{n-2} \\ \vdots & \vdots & \ddots & \vdots \\ v_1 & v_2 & \cdots & v_0 \end{pmatrix} \in \mathbb{F}_2^{n \times n}$$

The algebra of $n \times n$ circulant matrices over $\mathbb{F}_2$ is isomorphic to the algebra of polynomials in the ring $\mathcal{R}$

Definition 2.2 (Linear Codes)**.** A linear code $\mathcal{C}$ of lenght n and dimension k (denoted as $[n, k]$) is a subspace of $\mathcal{V}$ of dimension k; the elements of $\mathcal{C}$ are called *codewords*.
Due to the linearity of $\mathcal{V}$, any linear combination of two codewords is also a codeword.

Definition 2.3 (Generator Matrix)**.** Let $\mathcal{C} = [n, k]$ be a linear code. The matrix $\mathbf{G} \in \mathbb{F}_2^{k \times n}$ is called the generator matrix of $\mathcal{C}$ if

$$\mathcal{C} = \{\mathbf{mG}, \text{ for } \mathbf{m} \in \mathbb{F}_2^k\}$$

Definition 2.4 (Parity-Check Matrix)**.** Let $\mathcal{C} = [n, k]$ be a linear code. The matrix $\mathbf{H} \in \mathbb{F}_2^{(n-k) \times n}$ is called the parity-check matrix of $\mathcal{C}$ if

$$\mathcal{C} = \{\mathbf{v} \in \mathbb{F}_2^n \text{ such that } \mathbf{Hv}^\top = 0\}$$

In other words, $\mathbf{H}$ is the generator matrix of the dual code $\mathcal{C}^\perp$.

Definition 2.5 (Syndrome)**.** Let $\mathcal{C} = [n, k]$ be a code, $\mathbf{H}$ its parity matrix and $\mathbf{v} \in \mathbb{F}_2^n$ a word. The syndrome of $\mathbf{v}$ is defined as

$$\mathbf{S}(\mathbf{v}) = \mathbf{Hv}^\top$$

From this definition, a vector $\mathbf{v}$ is a codeword if and only if its syndrome is the zero vector.

Definition 2.6 (Quasi-Cyclic Codes)**.** Let $\mathbf{c} = (\mathbf{c}_0, \cdots, \mathbf{c}_{s-1}) \in \mathbb{F}_2^{sn}$ be a vector, where we consider each $\mathbf{c}_i$ as a vector in $\mathbb{F}_2^n$. A linear code $\mathcal{C} = [sn, k]$ is Quasi-Cyclic of index s if, for any $\mathbf{c} \in \mathcal{C}$, the vector obtained by applying a circular shift to every block $\mathbf{c}_i$ simultaneously is also a codeword.

2.3.1 Hard problems for cryptography

The problems we are interested in are all variants of the *decoding problem*, which requires to find the closest codeword to a given word with respect to a certain metric; in our case we consider as metric the Hamming weight, denoted as $\mathtt{wt}(\mathbf{v})$, which is the amount of non-zero coordinates of the vector $\mathbf{v}$. In the case of linear codes we have shown that the decoding problem stays the same whether one is given the syndrome of the received vector or

the vector itself; for this reason, we will only consider the *Syndrome Decoding* (SD) problem.

Definition 2.7 (SD Distribution)**.** Given $n, k, w \in \mathbb{N}$, the distribution $\mathsf{SD}(n, k, w)$ randomly samples a parity-check matrix $\mathbf{H} \xleftarrow{\$} \mathbb{F}_2^{(n-k)\times n}$, a vector $\mathbf{x} \xleftarrow{\$} \mathbb{F}_2^n$ such that $\mathtt{wt}(\mathbf{x}) = w$ and returns $(\mathbf{H}, \mathbf{S}(\mathbf{x}))$.

Definition 2.8 (SD Problem)**.** Given $(\mathbf{H}, \mathbf{y}^\top) \in \mathbb{F}_2^{(n-k)\times n} \times \mathbb{F}_2^{(n-k)}$ from the $\mathsf{SD}(n, k, w)$ distribution, the SD problem requires to find a vector $\mathbf{x} \in \mathbb{F}_2^n$ such that $\mathbf{S}(\mathbf{x}) = \mathbf{y}^\top$ and $\mathtt{wt}(\mathbf{x}) = w$.

This problem has been proven to be NP-complete in [2].

2.4. Hamming Quasi-Cyclic

Hamming Quasi-Cyclic is a code-based cryptosystem, that includes both a public-key encryption scheme and a key encapsulation mechanism. The KEM is built by applying the HHK transformation on the PKE, hence we will focus on the latter. HQC employs two different codes:

- A public decodable code $\mathcal{C}$ generated by $\mathbf{G}$ which can correct up to δ errors efficiently with $\mathcal{C}.\mathsf{Decode}()$.
- A random code with parity-check matrix $\mathbf{H} = [\mathbf{I}_n, \mathbf{circ}(\mathbf{h})]$

The idea at the core of the PKE is to encode the message and add random errors $(\mathbf{r_1}, \mathbf{r_2}, \mathbf{e})$, sparse polynomials with a certain Hamming weight that can be removed only by knowing the secret key: in this way the code $\mathcal{C}$ can decode the ciphertext only if these errors are removed. The authors showed that breaking the PKE scheme from a theoretical point of view is equivalent to solving the decisional version of the SD problem for quasi-cyclic codes, thus proving that the PKE is IND-CPA secure.

Algorithm 2: HQC PKE

Data: $\mathcal{R}_\alpha := \{x \in \mathcal{R} | \mathtt{wt}(x) = \alpha\}$,
$\xleftarrow{\$}$: uniform sampling from r.h.s

Function KEYGEN()
- $\mathbf{h} \xleftarrow{\$} \mathcal{R}$;
- $(\mathbf{x}, \mathbf{y}) \xleftarrow{\$} \mathcal{R}_\omega \times \mathcal{R}_\omega$;
- $\mathbf{s} \leftarrow \mathbf{x} + \mathbf{hy}$;
- $\mathcal{K}_{pri} \leftarrow (\mathbf{x}, \mathbf{y})$;
- $\mathcal{K}_{pub} \leftarrow (\mathbf{h}, \mathbf{s})$;
- **return** $(\mathcal{K}_{pri}, \mathcal{K}_{pub})$

Function ENCRYPT($\mathcal{K}_{pub}, \mathbf{m}$)
- $\mathbf{e} \xleftarrow{\$} \mathcal{R}_{\omega_e}$;
- $(\mathbf{r}_1, \mathbf{r}_2) \xleftarrow{\$} \mathcal{R}_{\omega_r} \times \mathcal{R}_{\omega_r}$;
- $\mathbf{u} \leftarrow \mathbf{r}_1 + \mathbf{hr}_2$;
- $\mathbf{v} \leftarrow \mathbf{mG} + \mathbf{sr}_2 + \mathbf{e}$;
- **return** $\mathbf{c} = (\mathbf{u}, \mathbf{v})$

Function DECRYPT($\mathcal{K}_{pri}, \mathbf{c}$)
- **return** $\mathcal{C}.\mathit{Decode}(\mathbf{v} - \mathbf{uy})$

3. Implementation

4. Experimental results

5. Conclusions

A final section containing the main conclusions of your research/study have to be inserted here.

References

[1] Nicolas Aragon, Paulo Barreto, Slim Bettaieb, Loic Bidoux, Olivier Blazy, Jean-Christophe Deneuville, Phillipe Gaborit, Shay Gueron, Tim Guneysu, Carlos Aguilar Melchor, Rafael Misoczki, Edoardo Persichetti, Nicolas Sendrier, Jean-Pierre Tillich, Gilles Zemor, Valentin Vasseur, Santosh Ghosh, and Jan Richter-Brokmann. BIKE Supporting Documentation. `https://bikesuite.org`, 2022.

[2] Elwyn R. Berlekamp, Robert J. McEliece, and Henk C. A. van Tilborg. On the inherent intractability of certain coding problems (corresp.). *IEEE Trans. Inf. Theory*, 24(3):384–386, 1978.

[3] Daniel J. Bernstein, Tung Chou, Carlos Cid, Jan Gilcher, Tanja Lange, Varun Maram, Ingo von Maurich, Rafael Misoczki, Ruben Niederhagen, Edoardo Persichetti, Christiane Peters, Nicolas Sendrier, Jakub Szefer, Cen Jung Tjhai, Martin Tomlinson, and Wen Wang. Classic McEliece Supporting Documentation. `https://classic.mceliece.org`, 2022.

[4] Ming-Shing Chen, Chen-Mou Cheng, Po-Chun Kuo, Wen-Ding Li, and Bo-Yin Yang. Faster multiplication for long binary polynomials. *CoRR*, abs/1708.09746, 2017.

[5] Shuhong Gao and Todd D. Mateer. Ad-

ditive fast Fourier transforms over finite fields. *IEEE Trans. Inf. Theory*, 56(12):6265–6272, 2010.

[6] Qian Guo, Clemens Hlauschek, Thomas Johansson, Norman Lahr, Alexander Nilsson, and Robin Leander Schröder. Don't reject this: Key-recovery timing attacks due to rejection-sampling in HQC and BIKE. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2022(3):223–263, 2022.

[7] Matthias J. Kannwischer, Richard Petri, Joost Rijneveld, Peter Schwabe, and Ko Stoffelen. PQM4: Post-quantum crypto library for the ARM Cortex-M4. `https://github.com/mupq/pqm4`.

[8] Wen-Ding Li, Ming-Shing Chen, Po-Chun Kuo, Chen-Mou Cheng, and Bo-Yin Yang. Frobenius additive fast fourier transform. In Manuel Kauers, Alexey Ovchinnikov, and Éric Schost, editors, *Proceedings of the 2018 ACM on International Symposium on Symbolic and Algebraic Computation, ISSAC 2018, New York, NY, USA, July 16-19, 2018*, pages 263–270. ACM, 2018.

[9] Sian-Jheng Lin, Wei-Ho Chung, and Yunghsiang S. Han. Novel polynomial basis and its application to reed-solomon erasure codes. In *55th IEEE Annual Symposium on Foundations of Computer Science, FOCS 2014, Philadelphia, PA, USA, October 18-21, 2014*, pages 316–325. IEEE Computer Society, 2014.

[10] Carlos Aguilar Melchor, Nicolas Aragon, Slim Bettaieb, Loıc Bidoux, Olivier Blazy, Jean-Christophe Deneuville, Philippe Gaborit, Edoardo Persichetti, Gilles Zémor, and IC Bourges. Hamming Quasi-Cyclic (HQC). `https://pqc-hqc.org/`, 2018.

[11] Thales Bandiera Paiva and Routo Terada. A Timing Attack on the HQC Encryption Scheme. In Kenneth G. Paterson and Douglas Stebila, editors, *Selected Areas in Cryptography - SAC 2019 - 26th International Conference, Waterloo, ON, Canada, August 12-16, 2019, Revised Selected Papers*, volume 11959 of *Lecture Notes in Computer Science*, pages 551–573. Springer, 2019.

[12] Joris van der Hoeven and Robin Larrieu. The frobenius FFT. In Michael A. Burr, Chee K. Yap, and Mohab Safey El Din, editors, *Proceedings of the 2017 ACM on International Symposium on Symbolic and Algebraic Computation, ISSAC 2017, Kaiserslautern, Germany, July 25-28, 2017*, pages 437–444. ACM, 2017.

[13] Guillaume Wafo-Tapa, Slim Bettaieb, Loïc Bidoux, Philippe Gaborit, and Etienne Marcatel. A practicable timing attack against HQC and its countermeasure. *Adv. Math. Commun.*, 16(3):621–642, 2022.